Blob and streak annotation of SSA imagery: assessment response

Pulkit | AI/ML Data Annotation Intern
Code: github.com/arch-pulkit/digantara-ssa-annotation
Tiles, labels and repatched images: https://drive.google.com/drive/folders/1fteqOhAg8GCEMrzDNjk_veAPt4rEI0vS?usp=sharing

Summary

I annotated the 10 FITS images with a semi-automatic pipeline. The code detects every source on the full image and classifies it as a blob or a streak from its shape. I then checked every streak, plus a band of objects the rules could not decide, by eye. After that the full images and their masks were cut into 70 tiles each (700 in total) and exported in YOLO segmentation format, with class 0 = blob and class 1 = streak.

There are 99,854 detections in total, and 59 of them are streaks. Drawing masks by hand at this scale was not realistic, so the manual work went into the ambiguous cases. All outputs can be regenerated from the raw FITS files plus one CSV file that holds my review decisions.

I had not worked with astronomical images before this assessment, so most of the choices below come from measuring the data first and looking at the result after each step.

1. The data

All 10 files are 9568 x 6380, stored as int16 with BZERO = 32768 (so the real range is 0 to 65535), with a 0.2 s exposure through 390 mm f/1.3 optics. Eight files have the OCS header with OPSMODE = static_staring. The two CAM_B files have a shorter header that does not state the mode. A few header values are placeholders (site at 10°/10° in the 15 August OCS files, pixel size -1), so I did not rely on those.

The pixel statistics split the files into three groups, and this ended up shaping most later decisions:

Group (files)	Sky level	Noise	Max value	Pixels exactly 0
A: 1a60, 4a8e, 4e2c, 7d7f, 7e5f	~1 ADU	~0.6 ADU	1001 (one star)	27%
B: 3e6b, 66ed, 7030	2-3 ADU	~2.5-3.5 ADU	4095 (12-bit cap)	31%
C: both CAM_B files	~43 ADU	~37-45 ADU	65535 (16-bit cap)	15%

Two things were not obvious from the brief. First, the images are not raw. Between 15% and 31% of pixels are exactly 0, which a real sensor does not produce because of its bias offset. So the sky had already been subtracted and negative values set to zero. Because the lower half of the noise is missing, sigma-clipped noise comes out too low (group C: 36.7 ADU, against about 45 ADU when measured from the upper half as p84 - median). Second, saturation is not the same everywhere. Group B stops at 4095, i.e. 12-bit data in a 16-bit file, so my first check for pixels equal to 65535 reported no saturation there, even though 18 pixels sit at the cap.

Assumption: stars are point sources in all 10 frames. With static staring and 0.2 s, the sky moves about 1.2 px during the exposure, if the pixels are 3.76 µm (about 2″/px). The pixel size is missing from the header, so this is an estimate. For the CAM_B files I assumed the same mode, and the round stars in those images are consistent with that.

2. Q2(a) Pre-processing

Between groups, the sky level differs by a factor of about 40 and the noise by about 70, so one fixed threshold in ADU could not work. For example, 20 ADU is about 30σ in group A but below the sky in group C. The goal of pre-processing was to put all images on the same scale without changing the pixel grid. Steps, in order:

1. **Convert to float32** before any arithmetic. In uint16, 1 - 3 wraps round to 65534.
2. **Background map:** 128 x 128 px boxes, sigma-clipped median in each box, a 3 x 3 filter across the boxes, then interpolation to full size. The box is much larger than a star but small enough to follow the vignetting. I used the median because the clipping only removed values below it.
3. **Noise map** from the same boxes, then (image - background) / noise. After this, the sky is close to 0 in every image and values are in units of σ.
4. **Detection only:** 5 x 5 Gaussian smoothing (FWHM 3 px, roughly the size of a star), then background and noise measured again on the smoothed image. Section 4 explains why the second measurement matters.
5. **Exported images only:** one 8-bit stretch per full image, never per tile, so the repatched images have no seams.

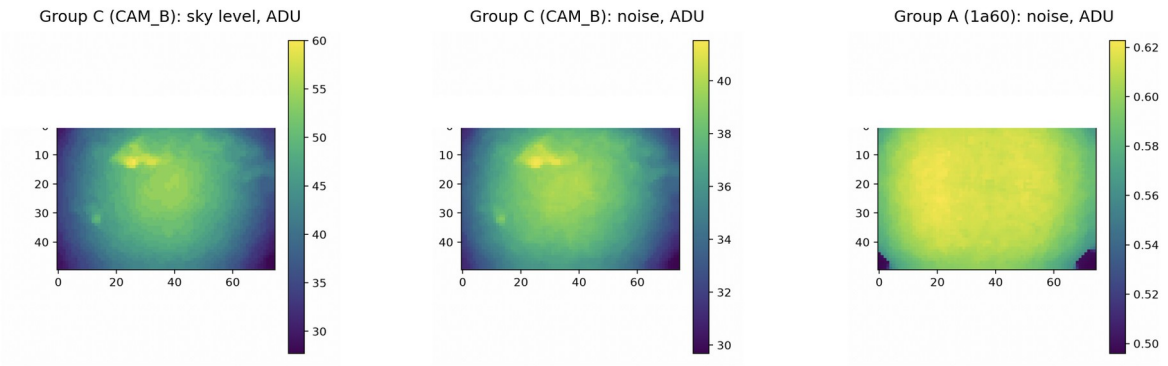


Figure 1. Background and noise per 128 px box. Group C has strong vignetting (sky 28-60 ADU) and a bright patch near the top left. In group A the sky was already flat, but the noise still follows the vignetting.

The group A noise map was the part I did not expect. Subtracting the sky level does not remove the photon noise that came with that sky, so the centre of the frame is still noisier than the corners. That is why the threshold follows a per-pixel noise map rather than a single number.

Steps I chose not to apply:

- Converting to 8-bit before detection. One 8-bit step would cover about 256 original values, and faint stars a few ADU above the sky would disappear.
- Median filtering to reduce noise. A 3 x 3 median deletes stars that are only 2-3 px wide. Noise and hot pixels are handled later by the size and shape checks.
- Resampling, rotating or flipping. The masks must line up with the original pixels (row 0 is the top of each tile).
- Dark/flat calibration. No calibration frames were provided, so the local background map deals with gradients and vignetting instead.

One problem showed up late, in the exported overlays. The median of integer data is itself an integer, so in groups A and B the background map is made of 1 ADU steps. After stretching, these showed up as large dark patches (in group A one step is almost 2σ). Detection was not affected, because it uses the background re-measured on the smoothed image, where values are continuous. I switched only the display images to a mean-based background.

3. Q2(b) Tiling without losing data

9568 = 9 x 1024 + 352 and 6380 = 6 x 1024 + 236, so each image needs 10 x 7 = 70 tiles. I compared two ways of handling the leftover strips:

- **Padding:** 16 of the 70 tiles would contain filler. The corner tile would hold only 352 x 236 real pixels, 8% of the tile, so a model would see large blank areas that never occur in real frames.
- **Shifting the last tile back (used):** the last column starts at 9568 - 1024 = 8544 and the last row at 6380 - 1024 = 5356, so they end exactly on the image edge. They overlap the previous tile by 672 px and 788 px. Every tile is 1024 x 1024 real pixels and nothing is resized. The cost is that objects in the overlap appear in two tiles (four in the corner). That is fine for training, since each tile should label everything visible in it.

Tile names carry their top-left position (<image>_r<row>_c<col>), so any tile can be put back in place. To check nothing was lost, I rebuilt each image from its tiles and compared every pixel with the original, along with a coverage map and a tile count. All 10 images matched. I then removed the edge rule on purpose to make sure the check can actually fail: it produced 54 tiles, and both the coverage and equality checks failed. My first check ("every tile is 1024 x 1024") still passed with zero tiles, which is why I added the tile count. In the final export, the repatched images are built from the PNG files read back from disk, so they also confirm the saved tiles.

Decision: detection and classification run on the full image, and tiling happens at the end. If each tile were processed on its own, a streak crossing a tile border would be split into two short pieces that could be classified as blobs.

4. Detection, and Q2(c) short/fat streaks vs blobs

Each detected source gets an ellipse fitted from its brightness-weighted second moments (the same idea as PCA on the pixel coordinates), and I use elongation = major axis / minor axis. My first idea was to measure how far bright pixels reach from the centre in each direction. That depends on the single farthest pixel, though, so one noise pixel or a neighbouring star changes the answer. The moment fit averages over all pixels, and it works at any angle. A plain x/y extent would not: a diagonal streak has a square bounding box.

I tuned the detection on one image (1a60), looking at cutouts of the most elongated objects after every change:

Attempt	What happened	What I changed
1. 3σ, no smoothing	2244 objects, 392 with elongation > 2. Tiny objects (5 px, ~0.2 px wide) had random elongations up to 7.5. Two faint streaks were split into halves 19-24 px apart.	Trust shape only above a minimum area. Smooth before detecting so faint streaks stay in one piece.
2. Smoothing + deblending	Deblending was meant to split touching star pairs, but all 6 objects it created were cut-up streaks.	Dropped deblending, and changed one thing at a time from then on.
3. Smoothing only	Streaks were whole, but the 3 faintest streaks and many faint stars were gone (1258 objects).	I was still comparing against the unsmoothed noise, which smoothing had reduced. Re-measured the noise after smoothing.
4. Re-measured noise, 5σ	3523 objects. All 6 streaks whole (area 133-345 px, elongation 6.2-12.0); the next object was at 4.2.	Streak rule: elongation ≥ 5 and area ≥ 30 px.

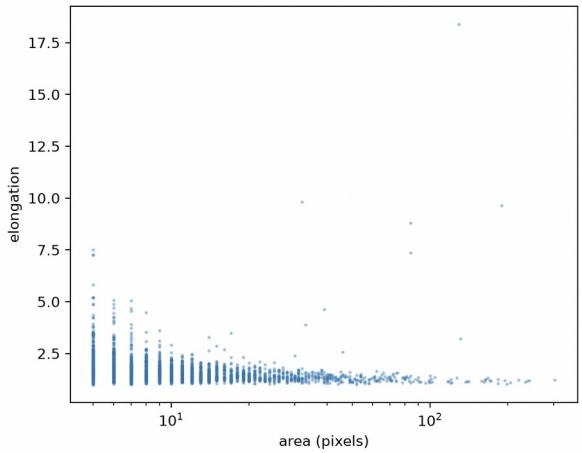


Figure 2. Attempt 1 on 1a60. Small objects scatter up to 7.5 purely from noise; the real streaks are the few points that are both large and elongated.

On 1a60 the gap was clear, and it held in the other group A images. In 66ed and the CAM_B files it disappeared. When I went through the objects just below the cutoff, I found real streaks the rule had missed. Bright streaks look wide because their blurred edges also pass the threshold, and this pulls their elongation down to 3.5-4.5. That is the same range as touching star pairs, which went up to 3.9:

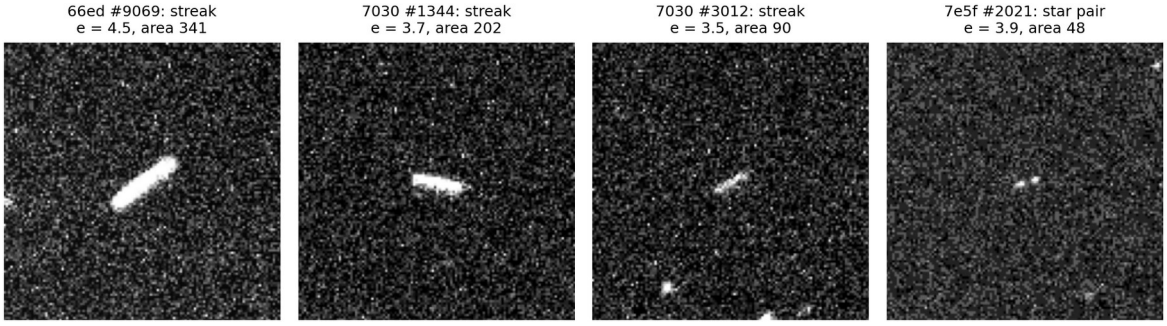


Figure 3. Three real streaks and one pair of touching stars, all at elongation 3.5-4.5.

So elongation alone cannot separate short/fat streaks from blobs. What separates them in the cutouts is the brightness along the object: a streak is evenly bright along its length, while two stars show two peaks with a dip between them. The final criteria are:

- **Streak:** elongation ≥ 5 and area ≥ 30 px. The area limit was 50 at first; I lowered it after a faint streak with area 49 was missed.
- **Manual review:** elongation 3-5 with area ≥ 50 px (big enough to judge), or elongation ≥ 5 with area 15-29 px. I checked these cutouts with the brightness criterion above, and I also checked every automatic streak.
- **Blob:** everything else with a valid shape.

My decisions are stored in annotations/manual_review.csv (image, label, verdict), and they override the automatic class.

Streak class, all 10 images	Result
Flagged automatically / confirmed by eye	51 / 50
Objects in the review band / streaks found there	98 / 9
Precision / recall of the automatic rule	50/51 (98%) / 50/59 (85%)
Final number of streaks	59

The one false positive was two faint stars that smoothing had joined into a thin dumbbell shape. Both kinds of error come from using shape without the brightness profile, so an automatic profile check along the major axis is the first thing I would add.

5. Q2(d) Criteria for faint or small blobs

A detection that is not a streak is kept as a blob if:

1. its pixels are at least 5σ above the local background of the smoothed image. The noise must be measured after smoothing. In attempt 3 I used the unsmoothed noise, which made the threshold several times stricter than intended: I got 1258 objects instead of 3523;
2. it has at least 5 connected pixels. At 3σ, noise alone puts about 0.13% of pixels above the threshold, roughly 80,000 per image, but they are scattered. Five touching noise pixels are rare, while a real star covers several pixels because of the PSF;
3. its unsmoothed flux gives a valid shape (positive total flux). Between 1 and 16 detections per image failed this and were dropped as noise;
4. saturated stars are blobs regardless of size, using each group's own ceiling (4095 or 65535).

The star-sized smoothing is what lets faint stars through: it averages out pixel-to-pixel noise but keeps signal that has the shape of a star. This fits the example annotation in the brief, which also marks faint stars.

Limitation: because clipped data makes the noise look smaller than it is (by about 20% in group C), the real threshold is lower in groups B and C. Those images have 8,000-21,000 blobs each, against 3,500-6,000 in group A, and most of the near-miss cutouts in group C showed only grain. So some faint blobs there are probably noise. A separate threshold per group, chosen by checking a sample of faint detections by hand, would fix this.

6. Results

Image	Group	Detections	Streaks	Rebuilt identical
1a600998	A	3,523	6	yes
4a8e6cd4	A	6,089	1	yes
4e2cdd92	A	6,116	3	yes
7d7fefcb	A	3,631	8	yes
7e5f3e1c	A	6,119	10	yes
3e6beceb	B	8,349	7	yes
66ed4268	B	16,668	2	yes
7030c2ac	B	8,647	9	yes
CAM_B …153133	C	20,949	8	yes
CAM_B …154244	C	19,763	5	yes
Total		99,854	59	10 of 10

7. Limitations and next steps

- Touching star pairs are one blob. The class is right but the outline is merged. I tried deblending, but it split streaks rather than pairs, so I left it out.
- YOLO polygons follow pixel centres, so they sit about 0.5 px inside the mask edge, and a few tiny outlines with fewer than 3 points were skipped. The thresholds were tuned and checked on this data only.
- Two things I could not explain: the DATERISE to DATEFALL gap is about 2 s while the exposure is 0.2 s, and the bright patch in the group C background could be a star halo, a reflection or cloud. Neither affected the annotation.

With more time I would add the brightness-profile check, set a per-group faint threshold, deblend only objects that are not line-like, and train a quick YOLO-seg baseline on the tiles to find where it disagrees with the labels. That is usually a fast way to catch annotation mistakes.

I used AI to write the code for the steps I learned and decided on while working through the domain. The analysis, the parameter choices, the checks and the manual review are my own.